

ARCHITETTURA vs IMPLEMENTAZIONE

Perché confonderle crea caos nei progetti software

Daniel Neagu — Software Architect & Senior Software Engineer

1. Introduzione

Una delle cause più frequenti di caos nei progetti software è la confusione tra **architettura** e **implementazione**.

Sono due livelli distinti, con obiettivi diversi e impatti diversi.

Quando vengono mescolati, il risultato è sempre lo stesso: sistemi fragili, decisioni incoerenti e difficoltà crescenti nel mantenere ed evolvere il software.

Questo documento chiarisce la differenza tra i due concetti e spiega perché mantenerli separati è fondamentale per costruire software solido e sostenibile.

2. Cos'è l'architettura

L'architettura è la **struttura del sistema**.

Definisce:

- i confini dei componenti
- le responsabilità
- le dipendenze
- i flussi principali
- i vincoli
- le decisioni stabili nel tempo

L'architettura risponde a domande come:

- Quali sono i moduli principali del sistema?
- Come comunicano tra loro?
- Quali responsabilità ha ogni componente?
- Quali sono i rischi e i trade-off?
- Come garantiamo chiarezza, manutenibilità e scalabilità?

L'architettura è il **perché** e il **cosa**.

3. Cos'è l'implementazione

L'implementazione è **il modo in cui realizziamo l'architettura**.

Include:

- linguaggi
- framework
- pattern locali
- strutture dati
- dettagli di codice
- ottimizzazioni puntuali

L'implementazione risponde a domande come:

- Quale libreria usiamo?
- Come strutturiamo questa classe?
- Qual è il pattern migliore per questo caso?
- Come ottimizziamo questa query?

L'implementazione è **il come**.

4. Perché confonderle è pericoloso

4.1 Decisioni incoerenti

Decisioni locali impattano globalmente senza una visione d'insieme.

4.2 Confini sfumati

I componenti iniziano a fare troppo o troppo poco.

4.3 Fragilità del sistema

Ogni modifica locale rischia di rompere parti lontane del sistema.

4.4 Debito tecnico crescente

Si ottimizza per il breve termine, non per la sostenibilità.

4.5 Conflitti tra team

Senza una cornice chiara, ogni scelta diventa opinione.

5. Esempi concreti

Esempio 1 — Microservizi che diventano mini-monoliti

Accade quando l'architettura è "microservizi", ma l'implementazione ignora i confini:

- dipendenze circolari
- logiche duplicate
- responsabilità confuse

Esempio 2 — Gateway che fa troppo

Un gateway dovrebbe essere un router.

Se inizia a fare:

- autenticazione
- business logic
- trasformazioni complesse

allora non è più un gateway: è un anti-pattern.

Esempio 3 — Wizard complessi senza flusso definito

Senza un'architettura chiara:

- stati
- transizioni
- responsabilità
- validazioni

l'implementazione diventa un patchwork di condizioni e hack.

6. Come mantenere separati architettura e implementazione

6.1 Documentare l'architettura

Diagrammi semplici, confini espliciti, responsabilità definite.

6.2 Stabilire principi non negoziabili

Nel mio lavoro:

chiarezza, manutenibilità, scalabilità.

6.3 Rendere stabili le decisioni architetturali

Non cambiano ogni settimana.

6.4 Lasciare libertà nell'implementazione

Finché non viola i principi architetturali.

6.5 Fare review a due livelli

- review architetturale
- review di implementazione

7. Conclusione

L'architettura è la visione.

L'implementazione è la realizzazione.

Confonderle significa costruire software fragile.

Separarle significa costruire software che vive bene nel tempo.

È un principio semplice, ma è la base di ogni progetto solido.

Applicazione pratica nel mio progetto

La distinzione tra architettura e implementazione non è solo teorica: è il principio che ha guidato la progettazione del mio backend di gestione economica personale. L'architettura definisce i confini dei moduli (Identity, Movimenti, Categorie, Reporting, Security, Otp), la separazione dei layer (API, Application, Domain, Infrastructure) e le interazioni cross-modulo tramite eventi. L'implementazione, invece, riguarda le scelte locali: DTO, repository, query SQL, pattern dei service, ottimizzazioni puntuali.

Questa separazione ha evitato problemi tipici dei sistemi non strutturati: logica di business nei controller, entità di dominio usate come DTO, repository che contengono regole, moduli che si chiamano tra loro direttamente.

L'architettura ha stabilito confini chiari; l'implementazione li ha rispettati.

Un esempio concreto è il flusso Movimenti → Reporting: l'architettura stabilisce che i moduli non devono dipendere direttamente tra loro. L'implementazione realizza questo principio tramite un EventBus e un handler dedicato, evitando accoppiamento diretto e mantenendo il sistema estensibile.

La distinzione tra architettura e implementazione permette anche di scalare il lavoro in team: l'architettura definisce i confini e le responsabilità, mentre l'implementazione lascia libertà ai singoli sviluppatori finché non viola i principi architetturali. Questo riduce conflitti, facilita l'onboarding e rende il sistema governabile nel tempo.

Come prendo decisioni architetturali

Le decisioni architetturali sono stabili e guidate da tre principi non negoziabili: chiarezza, manutenibilità, scalabilità.

Una decisione è architetturale quando modifica i confini, le responsabilità o i flussi del sistema. Una decisione è implementativa quando riguarda dettagli locali che non impattano la struttura.

Per ogni decisione valuto:

- impatto sul dominio
- impatto sui moduli
- impatto sul team
- impatto sulla scalabilità futura
- rischi e trade-off
- alternative possibili

Questo approccio evita scelte impulsive e garantisce coerenza nel tempo.